A Cross Sectional Ranking System Using Neural Networks and Hierarchical Risk Parity

Daniel Paxton[1]

[1]Quantitative Research Dept. Michigan Algorithmic Traders

August 19, 2026

Abstract

This paper aims to showcase a novel Machine Learning-based trading pipeline using a Neural Network for rank regression and Hierarchical Risk Parity for robust portfolio allocation. We attempted to use a mean-based target framework with the log-relative returns of the S&P 500 for engineering our main features. This approach aims to isolate idiosyncratic information to utilize as features in our Neural Network.

1 Introduction

AI (specifically Gemini 3.1 Pro) was used to assist in the creation of the code for this project (first draft). However, the codebase has been thoroughly reviewed and reflects the thinking and design choices of the author.

A common challenge in quantitative asset management is the separation of idiosyncratic risk and systemic risk. By utilizing features to model regimes and trend, we pursued a robust, risk-managed framework that extracts alpha from large-cap US equities.

In highly efficient markets, like large-cap US equities, consistently outperforming the benchmark is difficult, especially when it comes to traditional machine learning methods. The objective of this research does not claim the discovery of a significant alpha source, however, the pipeline used can be repurposed in other asset classes or with different models.

Our strategy employs a Neural Network regressor in order to learn the cross sectional relationships in stocks regarding their returns compared against the SPY. We then rank the model outputs This is done by sorting the output of our neural network regressor and creating a basket of stocks of a designated length. Our rationale behind the use of Neural Networks is their ability to model complex nonlinear functions. In our use case, it may be important to have the information of the whole tradable universe in order to make predictions. Finally, we then feed the outputs of the Neural Network model into Hierarchical Risk Parity [**prado2018advances**].

2 Why the MLP-HRP Hybrid Structure?

If we were to equally weight the stocks ranked from our Neural Network, we would run into a major problem. If we were to equally rank these stocks, we cannot effectively diversify our portfolio. The job of the model is to see which stocks will outperform the SPY, relative to other stocks, and this can be skewed by many factors. This can create clusters in our model predictions that an equal weighting scheme will not diversify. For example our model may predict that the technology sector will outperform every other sector, thus predicting that technology stocks will outperform thus putting it in our portfolio. In addition, another problem arises, we assume that our regressor model will output higher numbers if it is more confident in a specific stock outperforming, and we should not treat this equally compared to the other stocks. Due to the complex nature of our Neural Network we expect a very small IC (Information Coefficient) when testing out of sample. That is expected and that makes our portfolio allocation model much more important. We can still extract a meaningful equity curve and take advantage of a small IC with a allocation model. It is very difficult to solve both problems efficiently at the same time. We chose to focus mainly on the first problem, that we may expose ourselves to high factor risk in an equal weighting model. We fix this by introducing a robust price aware machine learning model that trains on the correlation matrix of our portfolio basket, this is Hierarchical Risk Parity. This allocation model should be efficient enough to "utilize" the information extracted from our Neural Network.

3 Universe Selection

4 Feature Engineering

In order to feed our neural network with quality features, we created a custom feature matrix with various volatility adjusted momentum indicators (brainstormed with the assistance of AI). When it comes to our cross sectional price ranking model, it makes sense to mostly use indicators that are based on individual momentum over trend indicators. However, we still utilize trend indicators in our feature matrix. Our first feature is simply log daily returns denoted as

$$M_{i,t} = \ln(P_{i,t}) - \ln(P_{i,t-1})$$

One of the most common features in US equities for machine learning models is stock market returns. Due to the noisiness of such features, we define a set of standard deviation adjusted returns over the lookback periods n = 5, 20, 252 where we use a 20-day lookback for calculating the standard deviation for 5 and 20 day indicators. The 252 day lookback for returns utilizes a 60 day standard deviation. These serve as the short term and long term volatility adjusted returns indicators.

$$M_{i,t,n} = \frac{\ln(P_{i,t}) - \ln(P_{i,t-n})}{\sigma_{i,t,m}}$$

When it comes to returns of assets, volatility regimes are crucial in determining anomalies. Volatility regimes are defined as a stable volatility over a given period. This can be calculated at the market level and at the individual series in our tradable universe. In order to estimate a break in the current regime we created the Volatility Ratio as follows:

$$VR_{i,t} = \frac{\sigma_{i,t,20}}{\sigma_{i,t,60}}$$

As this ratio increases, we expect the underlying true regime to follow. Because of the nature

of rolling features, it is expected that it will lag behind the true regime. We chose lookback periods of 20 and 60 in order to reduce noise and lag while still capturing the underlying regime shifts.

In addition to our volatility features, we added additional features that calculate the normalized distance from our market benchmark, the SPY. This allows us to analyze the asset's behavior with respect to the overall market.

$$D_{i,t,20} = \frac{\ln(P_{i,t-20}) - ln(P_{SPY,t})}{\sigma_{i,t,20}}$$

$$D_{i,t,60} = \frac{\ln(P_{i,t}) - ln(P_{SPY,t})}{\sigma_{i,t,60}}$$

Similarly we added a feature to capture over extensions and exhaustion of price movement.

$$D_{i,t,60} = \frac{\ln(P_{i,t}) - ln(P_{i,t-60})}{\sigma_{i,t,60}}$$

$$D_{i,t,20} = \frac{\ln(P_{i,t}) - ln(P_{i,t})}{\sigma_{i,t,20}}$$

To capture industry based group behaviors, we utilize a one-hot encoded Morningstar Sector Code for each stock provided by Quantconnect. By one hot encoding the variables we are able to feed it into our neural network.

5 Target Variable: Idiosyncratic Excess Returns

To ensure that the neural network is extracting cross sectional relationships rather than systematic relationships, we employ a cross sectional percentile ranking target that measures idiosyncratic excess returns. This approach attempts to solve the novel problem of using neural networks for raw price prediction.

$$\alpha_{i,t} = (\ln(P_{i,t+33}) - \ln(P_{i,t})) - (\ln(P_{SPY,t+33}) - \ln(P_{SPY,t}))$$

We must compute this for every timestep, and in our case we computed this for each day. To address the varying scales for different regimes, we assign percentiles to everything in our tradable universe depending on how much they outperformed the SPY. Higher percentile stocks indicate stocks that outperformed the SPY relative to the other stocks. This makes our target variable stationary without any preprocessing. After ranking stocks we can represent our targets such that:

$$y_{i,t} = Rank_{pct}(\alpha_{i,t}) \in [0, 1]$$

This method ensures that the Neural Network does not have to worry about the magnitude of an asset's performance against our benchmark, but rather the relative cross sectional ranking in our tradable universe.

6 Model Architecture

Our main predictive engine is a standard Neural Network. Given a dense selection of features, we must be meticulous with our preprocessing due to the low signal-to-noise ratio. We created this architecture with simplicity in mind.

The model consists of two hidden layers with the following configurations:

- Preprocessing: Principal Component Analysis with preserving 95% variance
- Hidden Layer 1: 64 Neurons
- Hidden Layer 2: 16 Neurons
- Activation Function: Rectified Linear Unit (ReLU)
- Optimization Algorithm: Adam (Adaptive Moment Estimation)
- Maximum allowed iterations: 500 (This is to prevent overfitting)
- Regularization (L2): 0.1. This term is the magnitude of the squared weights that are

added to the loss function to prevent overfitting

7 Portfolio Allocation

The final stage of our model utilizes **López de Prado's** [**prado2018advances**] **Hierarchical Risk Parity (HRP)** framework to perform capital allocation. After our Neural Network returns a top N rank of securities, we utilize HRP in order to safely determine weights. We chose HRP because it avoids the pitfalls of Mean-Variance Optimization by relying less on the covariance matrix of our assets.

The algorithm computes a distance matrix D based on the correlation matrix ρ of the top N ranked assets from our Neural Network. The distance between assets i and j is denoted as:

$$d_{i,j} = \sqrt{0.5(1 - \rho_{i,j})}$$

With our distance matrix we use Ward Variance linkage in order to create a hierarchy. The Ward method is the most effective when it comes to creating more variance-uniform clusters. For each iteration we calculate the total within-variance cluster sum denoted as:

$$d(u, v) = \sqrt{\frac{|u||v|}{|u| + |v|}} \|\bar{u} - \bar{v}\|_2$$

Where $|u|$ and $|v|$ are the lengths of each cluster and $\bar{u}$ and $\bar{v}$ represent the mean of the cluster. This metric gives us a score indicating the variance of the cluster if we were to merge them. We must iterate through each of our clusters to see which pairs we can merge in order to create the lowest variance cluster. The rationale behind this is that among the set of possible pairwise within-variance sums, we merge the cluster that will create the least variance, thus reducing the amount of variance in our portfolio when we allocate. Since this creates a hierarchy, we can assume that clusters share similar price behavior, potentially leading to groups that are influenced by the same factors.

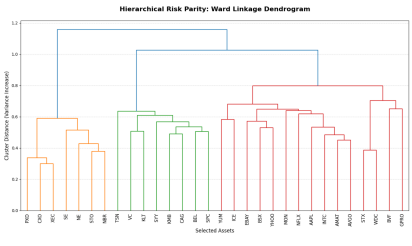

Figure 1: Example of a hierarchy in equities

With our hierarchy structure, the final stage of this allocation algorithm is to assign weights to each asset. We assume that the sum of asset proportions adds to 1, indicating that we fully exhaust the budget. Rather than bisecting each branch, we ignore the splits in the dendrogram. We use the linkage algorithm to simply get a one-dimensional organized array of stock tickers. By recursively splitting the array in half and forming clusters, we allocate more capital to assets that are somewhat "independent" from the other assets. López de Prado [**prado2018advances**] introduces an inverse variance weighting scheme, to further allocate to "unique" stocks. This algorithm can now allocate stocks based off of their empirical variance and their relation to other stocks in the universe. The main benefit of HRP is the lack of an inverse covariance matrix, which tends to be extremely noisy. We can define the amount of capital we should allocate to the left cluster as:

$$\alpha_{left} = 1 - \frac{V_L}{V_L + V_R}$$

And the capital assigned to the right cluster is:

$$\alpha_{right} = 1 - \alpha_{left}$$

Figure 1 shows what a potential hierarchy can look like. The vertical axis represents the

As the variance of one cluster rises, we allocate less of our capital to that cluster. This is performed over every asset from our Neural Network. After allocating our budget, we have

created a risk parity across our asset clusters.

8 Live Trading Rules

Now that we have laid out the pipeline of our model, we must arrange a backtest to simulate trades on historical data. Originally, we were going to introduce an expanding window for the Neural Network; however, this can make backtests only accurate towards the end. This can cause a problem regarding the effectiveness of our model before deployment. Although that can be beneficial if we pretrain the model before starting the backtest, it can introduce too much variance in predictions in recent data. This is due to the architecture of a simple feedforward neural network. Essentially, we hope to model recent price regimes, and simple neural networks are unable to effectively model the long term temporal nature of financial markets. We aid this by creating a rolling window in which the model trains. We do not want to introduce a new architecture like RNNs or LSTMs due to the model complexity. More complex architectures typically require more training data and would essentially require a large rolling window or an expanding window, thus cutting down on our usable data. Using our Neural Network with a rolling window can also pose problems. It is very likely that our model will be unable to capture the complex nature of stock data, regardless of how many features we use. We hope that instead of attempting to model the chaotic, individual nature of each asset in our universe, we can capture the cross sectional relationships of stocks, specifically being the most confident in our top N stocks.

9 Results and Conclusion

Utilizing the Quantconnect platform, we performed a backtest starting from 2010-01-01 to 2026-01-01. Our top N stocks was set to 30. We achieved these results.

Metric	**Value**
Sharpe Ratio	0.518
Drawdown	26.500%
Loss Rate	44%
Win Rate	56%
Beta	0.84
Total Orders	10213
Total Fees	93410.69
Start Equity	100000
End Equity	5584894.84

Table 1: Strategy Performance Metrics

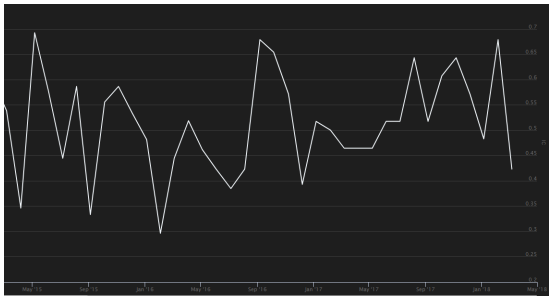

Figure 2: Truncated chart of our Information coefficient over time.

The Information Coefficient (IC) was used to measure the predictive power of our Neural Network. After each day we calculate the IC using this formula:

$$IC_t = 1 - \frac{6\sum_{i=1}^{n} d_{i,t}^2}{n(n^2-1)}$$

The IC will give us a score between -1 and 1, which specifically indicates the correlation between our predictions and the target variables. In addition, we calculated the accuracy of the top model picks, the ones that we traded. From our backtest we achieved the rolling results below:

10 Conclusion

In this paper, we aimed to evaluate a Neural Network-HRP pipeline aimed at capturing a sort of synergy effect potentially producing alpha. While the results successfully generated

an ample return when compared to a simple buy and hold strategy, the metrics are very similar. This gives us more insight into the pitfalls of rank based prediction in financial time series. With a beta of 0.84, our model failed to truly capture idiosyncratic risk; rather, our Neural Network was likely choosing non powerful signals, acting more like a random sampler of a specific industry. Since our beta is not far from 1, we can conclude that using a Neural Network in this type of pipeline can actually be detrimental to risk adjusted returns. This is potentially due to how our selection model will simply choose non diversified stocks. Due to the nature of how our model selected which stocks to trade, no risk model will be able to diversify a concentrated tradable asset list. We can prove this by increasing our top N. If the results are similar, that must indicate that our asset selection model is choosing highly clustered stocks, making our risk model obsolete. Using data from 2010-01-01 to 2026-01-01 and increasing our top N to 100 yields these results We have a beta and drawdown that is almost

Metric	**Value**
Sharpe Ratio	0.457
Drawdown	27.400%
Loss Rate	45%
Win Rate	55%
Beta	0.832
Total Orders	10185
Total Fees	86089.44
Start Equity	1000000
End Equity	4707523.58

Table 2: Strategy Performance Metrics

identical to our previous run. This indicates that our Neural Network ranking model makes it difficult to fully diversify assets. This is evidence for our Neural Network simply selecting stocks randomly in our universe. Rather than our selection model supporting the HRP model like we hoped it actually became a bottleneck.

Ultimately, we found that, although deep learning architectures possess the ability to map complex financial data, their predictions are unable to effectively model cross sectional idiosyncratic risk. When it comes to future models, it may be best to start with low parameter models in order to test for feature efficacy, then scale up from the baseline.

López de Prado, Marcos. *Advances in Financial Machine Learning.* Hoboken, NJ: John Wiley & Sons, 2018.